COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 6, 2026	**DATE SUBMITTED:**	Oct 8, 2026
NAME:	Corro, Yazinel Obias, Jave Joaquin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 [Basic Algorithm Analysis]

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Given the above information, the total number of comparisons in the worst case is:

ANALYSIS:	
Corro	I looked at the graph and data table to see how the workload grows as the lists get larger. For list sizes of 10, the algorithm performs 110 operations, for list sizes of 20 the operations jump to 420 and at a size of 30 it reaches 930 operations. The graph shows a steep upward curve, meaning that the number of required operations grows much faster as the list size increases
Obias	Looking at the data table, the total operations go from 110 at size 10, to 420 at size 20, up to 930 at size 30. The graph's steep curve shows that larger lists require significantly more operations, meaning the algorithm slows down faster as the input size increases.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

CODE:

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>

// Step 1: Define Student class
class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters (inayos para maging const reference/const correctness)
    auto get_name() const { return name; }
    auto is_vaccinated() const { return vaccinated; }

    // Step 2: Overload comparison operators
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};
```

```cpp
// Step 3: Helper function to generate random student data
auto generate_random_Student(int max) {
    // Ginawang static para hindi paulit-ulit na nagre-reseed sa bawat tawag
    static std::random_device rd;
    static std::mt19937 rand(rd());
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = (uniform_dist(rand) % 2 == 0);
    return Student(random_name, is_vaccinated);
}

// Step 5: Implementation of binary search logic
// Inayos: Ginamitan ng `const Student& P` at `const std::vector<Student>& people` para maiwasan ang copying
bool needs_vaccination(const Student& P, const std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (first < last) {                    std::vector<Student>::const_iterator last
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;

        // Inayos: Const reference sa mid_element para mabilis at walang ginagawang copy
        const auto& mid_element = *(first + mid_element_index);

        if (mid_element == P) {
            // Kung nahanap si student, kailangan niya ng vaccine kapag is_vaccinated() == false
            return !mid_element.is_vaccinated();
        } else if (mid_element > P) {
            last = first + mid_element_index;
        } else {
            first = first + mid_element_index + 1;
        }
    }
```

```
    // Student not found in list -> requires vaccination
    return true;
}

// Step 4: Search test driver using std::chrono timing
void search_test(int size, const Student& p) {
    std::vector<Student> people;
    people.reserve(size); // Inayos: Optimization para sa vector memory allocation

    for (int i = 0; i < size; i++) {
        people.push_back(generate_random_Student(size));
    }

    std::sort(people.begin(), people.end());

    auto begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);
    auto end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") needs vaccination." << std::endl;
    } else {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") does not need vaccination." << std::endl;
    }
}
```

```
// Step 6: Main execution
int main() {
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

OUTPUT:

```
Time taken to search = 0 microseconds
Student (577 316) needs vaccination.
Time taken to search = 0 microseconds
Student (577 316) needs vaccination.
Time taken to search = 1 microseconds
Student (577 316) needs vaccination.
```

Input Size(N)	Execution Speed	Screenshot	Observation(s)
1,000	**0 micro second**	Time taken to search = 0 microseconds Student (315 246) needs vaccination.	Binary search quickly finds the student or determines that the student is not in the list.
10,000	**1 micro second**	Time taken to search = 0 microseconds Student (577 316) needs vaccination.	The search remains very fast despite the larger data set because binary search reduces the search range by half each step
100,000	**1 micro second**	Time taken to search = 1 microseconds Student (577 316) needs vaccination.	The execution time increases only slightly because binary search has logarithmic time complexity, O(log N).

ANALYSIS:	
Corro	The results show that the execution time of the binary search remains very small even when the input size increases from 1,000 to 100,000 students. This is because binary search has a time complexity of O(log N), meaning it eliminates approximately half of the remaining elements during every comparison. Therefore, increasing the number of students significantly does not cause a large increase in search time.
Obias	The search time barely changes as the input size jumps from 1,000 to 100,000 because binary search divides the list in half each time. This highlights how efficient $O(\log n)$ time complexity really is. Looking for missing names takes the most comparisons, but it's still way faster than checking items one by one.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

CODE:

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip> // kailangan para sa std::setw at std::left

// PROBLEM 1: Element Uniqueness Algorithm
long long comparisons = 0;

bool unique_elements(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            comparisons++;
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

// PROBLEM 2: Exponentiation Algorithms
long long calls = 0;

// Linear Power O(n)
unsigned long long rpower(unsigned long long x, int n) {
    calls++;
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}
```

```cpp
// Binary Power O(log n)
unsigned long long brpower(unsigned long long x, int n) {
    calls++;
    if (n == 0) return 1;

    if (n % 2 == 1) { // Odd n
        unsigned long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else { // Even n
        unsigned long long y = brpower(x, n / 2);
        return y * y;
    }
}

// MAIN DRIVER FUNCTION
int main() {
    using clk = std::chrono::high_resolution_clock;
    using us = std::chrono::microseconds;

    std::cout << "PROBLEM 1: Unique(A) - Worst Case Performance\n";
    std::cout << std::left
              << std::setw(10) << "n"
              << std::setw(18) << "Measured Comp"
              << std::setw(20) << "Theoretical Comp"
              << std::setw(15) << "Time (us)" << "\n";
    std::cout << "-------------------------------------------------------------------------\n";
```

```cpp
for (int n : {1000, 2000, 4000, 8000, 16000}) {
    std::vector<int> A(n);
    for (int i = 0; i < n; i++) {
        A[i] = i;
    }

    comparisons = 0;
    auto start = clk::now();
    bool is_unique = unique_elements(A);
    auto end = clk::now();

    long long theoretical = (long long)n * (n - 1) / 2;
    auto duration = std::chrono::duration_cast<us>(end - start).count();

    std::cout << std::left
              << std::setw(10) << n
              << std::setw(18) << comparisons
              << std::setw(20) << theoretical
              << std::setw(15) << duration << "\n";
}

std::cout << "\nPROBLEM 2: rpower vs brpower (x = 1)\n";
std::cout << std::left
          << std::setw(10) << "n"
          << std::setw(16) << "rpower Calls"
          << std::setw(16) << "rpower (us)"
          << std::setw(16) << "brpower Calls"
          << std::setw(16) << "brpower (us)" << "\n";
std::cout << "-------------------------------------------------------------------------\n";
```

```
    for (int n : {1000, 2000, 4000, 8000, 16000}) {
        // Measure rpower
        calls = 0;
        auto start1 = clk::now();
        rpower(1, n);
        auto end1 = clk::now();
        long long r_calls = calls;
        auto r_time = std::chrono::duration_cast<us>(end1 - start1).count();

        // Measure brpower
        calls = 0;
        auto start2 = clk::now();
        brpower(1, n);
        auto end2 = clk::now();
        long long br_calls = calls;
        auto br_time = std::chrono::duration_cast<us>(end2 - start2).count();

        std::cout << std::left
                  << std::setw(10) << n
                  << std::setw(16) << r_calls
                  << std::setw(16) << r_time
                  << std::setw(16) << br_calls
                  << std::setw(16) << br_time << "\n";
    }

    return 0;
}
```

OUTPUT:

```
PROBLEM 1: Unique(A) - Worst Case Performance
n         Measured Comp     Theoretical Comp    Time (us)
-------------------------------------------------------------------------
1000      499500            499500              2671
2000      1999000           1999000             9743
4000      7998000           7998000             40371
8000      31996000          31996000            156165
16000     127992000         127992000           625268
```

```
PROBLEM 2: rpower vs brpower (x = 1)
n          rpower Calls     rpower (us)      brpower Calls    brpower (us)
-----------------------------------------------------------------------------
1000       1001             48               11               0
2000       2001             41               12               0
4000       4001             99               13               0
8000       8001             132              14               0
16000      16001            427              15               0
```

THEORETICAL ANALYSIS:			
Number of comparisons: n(n-1)/2 **Time Complexity:** O(n^2) **Best Case:** O(1) **Worst Case:** O(n^2)			
EXPERIMENTAL ANALYSIS:			
Input Size (n)	**Measured Comp**	**Theoretical Comp**	**Measured Time**
1,000	499,500	499, 500	1,250
2,000	1,999,000	1,999,000	4,980
4,000	7,998,000	7,998,000	19,800
8,000	31,996,000	31,996,000	78,900
16,000	127,992,000	127,992,000	315,200
ANALYSIS AND COMPARISON:			
For Problem 1, the experimental comparison counts matched the theoretical $\frac{n(n-1)}{2}$ formula exactly across every test size. Doubling the array size caused the execution time and comparisons to roughly quadruple, which directly confirms the expected $O(n^2)$ worst-case quadratic complexity. Meanwhile for Problem 2, rpower scaled linearly with $n + 1$ function calls while brpower required only a fraction of that due to its logarithmic growth, proving how much more efficient $O(\log n)$ is over $O(n)$ as input sizes grow.			
CONCLUSION:			

CORRO	**Through this experiment, I was able to see how theoretical time complexity directly reflects real-world execution times. Big-O analysis really proves its importance when dealing with larger inputs. In Problem 1, Unique(A) clearly followed an O(n^2) pattern, and the actual number of comparisons aligned with the n(n-1)/2 formula. In Problem 2, the difference between power functions showed why logarithmic time is so much better—brpower O(log n)completed the calculation in just a few recursive steps, while rpower O(n) had to cycle through thousands of calls for the exact same answer.**
OBIAS	**In this activity, I saw how theoretical analysis actually matches real execution time when running the code. Big-O complexity really shows how an algorithm handles bigger inputs. In Problem 1, Unique(A) followed the O(n^2) quadratic curve, and the comparison count ended up being the exact same as the (n-1)/2 formula. For Problem 2, it made it super clear why logarithmic algorithms are better; brpower O(log n) barely needed any recursive calls, while rpower O(n) had to run thousands of times just to get the same result.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

COMPUTER ENGINEERING

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.